

Image processing project documentation

Martyna Sadowska, Hanna Szczerbińska

1 Introduction

Our image-processing application supports a range of operations on imported images. It allows users to perform grayscaling, binarization, brightness and contrast adjustments, alongside graphic filters for blurring, sharpening, and edge detection -all utilizing various algorithms. The software supports morphological operations, image inversion, and histogram equalization. Apart from that, it displays real-time brightness histograms and spatial projections. After applying the chosen modifications, the processed image can be easily exported and saved locally.

The project is a desktop application implemented in Java. The Graphical User Interface is built using Swing, which provides a reliable, native toolkit. To ensure a modern appearance, the FlatLaf (FlatDarkLaf) library was utilized to apply a dark theme. Additionally, JFreeChart was integrated to efficiently generate and display the histograms. The complete source code is available on GitHub.

2 Available functionalities

Figure 1 shows the original photo used later to demonstrate the available functionalities.



Figure 1: Original photo used for testing.

2.1 Import and save

The import option allows the user to import a .jpg, .png, or .bmp file to edit in the application. Upon clicking on the 'import' button, a new file explorer window opens, which allows the user to navigate through directories and choose any file from their disk.

The user can also save the processed image onto their local device disk, using the 'save' option; upon clicking it, a file explorer window opens, allowing the user to choose a name and format for the file and the directory for it to be saved into.

2.2 Histograms and projections

2.2.1 Histograms

In the 'Display' menu, the user can toggle between displaying the overall brightness histogram and separate RGB histograms for each color channel. These histograms update automatically with every modification made to the image. They are generated by counting the number of pixels at each intensity level (either within a specific color channel or as the average of the red, green, and blue values for the overall brightness histogram). The overall brightness histogram is illustrated in Figure 2, while the RGB histograms are shown in Figure 3.

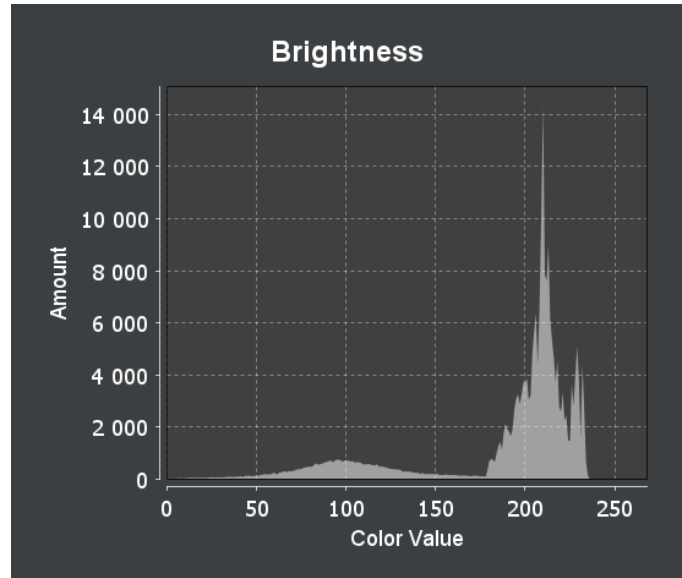


Figure 2: Brightness histogram of the original photo. The X-axis represents the brightness level, and the Y-axis represents the pixel count at that intensity.

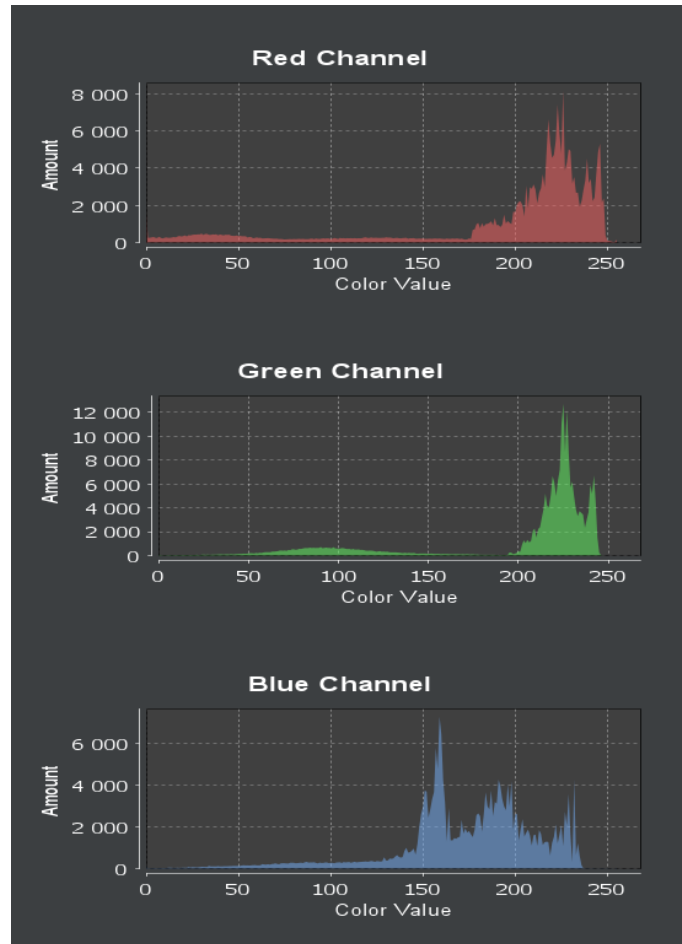


Figure 3: RGB histograms of the original photo. The X-axis represents the brightness level, and the Y-axis represents the pixel count at that intensity for each channel.

The histogram panel also allows for histogram equalization, which can be applied by clicking the corresponding button. This algorithm enhances the image's global contrast by spreading out the most frequent intensity values. To prevent color distortion, the image is first converted from the RGB to the HSV color space. The algorithm calculates the cumulative distribution function of the image's brightness and uses it to generate a table to map the original brightness values to a new, uniformly distributed range. After that, the modified HSV pixels are converted back to RGB. The results of this operation are shown in Figure 4.



Figure 4: Photo after histogram equalization.

2.2.2 Projections

In the 'Display' menu the user can toggle the visibility of vertical and horizontal projections around the image. These projections graphically display the distribution of black pixels along the image's rows and columns. The best results are achieved after binarizing the image. Figure 5 illustrates the horizontal and vertical projections generated for an image binarized via Otsu's method (detailed later in section 2.4.5).

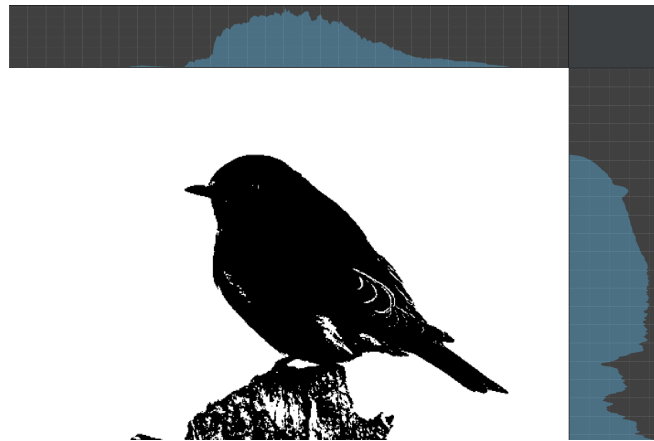


Figure 5: Projections around a photo binarized using Otsu's method.

2.3 Undo and revert

Undo and revert to the original state are basic methods available under the 'Edit' menu. Undo restores the image to its previous state (1 step back) and can be triggered by Ctrl+Z. The 'Revert to the original state' option restores the image to its initially imported, unmodified state.

2.4 Pixel operations

Pixel operations process an image by modifying the value of each pixel individually. These methods are primarily used to adjust the image's global visual characteristics, including color, luminance, and contrast. The available tools include grayscaling, negative inversion, brightness and contrast adjustments, and binarization. All of those operations can be accessed via the 'Pixel operations' option in the 'Edit' menu.

2.4.1 Grayscale

The grayscale panel allows for choosing one of the six available algorithms (averaging, luminance, destauration, decomposition, single color channel and custom) for converting a color image into shades of gray, which is triggered by clicking on the 'Apply' button. The algorithm takes values of red, green and blue on each pixel, calculates the gray value and assigns it to each of the color channels. The only difference between the algorithms is how the gray is calculated. Figure 6 shows examples of photos grayscaled using the algorithms.

Available algorithms:

- **Averaging** - gray is the average of red, green and blue values.

- **Luminance** - gray is the weighted average of red, green and blue, where the coefficients are taken from BT.601 recommendation: 0.299 for red, 0.587 for green and 0.114 for blue. This accounts for the fact that cone density in the human eye is not uniform across colors, so the conversion seems more accurate.
- **Desaturation** - gray is calculated by finding the midpoint between the maximum of (R, G, B) and the minimum of (R, G, B). This conversion results in a flatter, softer grayscale image.
- **Decomposition** - gray is either the maximum or the minimum of (R, G, B), depending on which option is chosen. A maximum decomposition provides a brighter grayscale image, while a minimum decomposition provides a darker one.
- **Single color channel** - gray is the value of one of the color channels, which can be chosen on the panel.
- **Custom** - this algorithm allows the user to specify how many shades of gray the resulting image will use (from 2 to 256, where 2 results in a black-and-white image, while 256 yields an image identical to averaging). The algorithm for calculating the gray value is as follows

$$\text{gray} = \text{round} \left(\frac{\text{avgVal}}{\Delta} \right) \cdot \Delta,$$

where *avgVal* is the gray value from the averaging algorithm and Δ is defined as the full scale of brightness (255) divided by the number of intervals between the available shades.

- **Custom with dithering** - this algorithm can be enabled alongside the 'Custom' option and uses the exact same base calculation. The key difference is that it additionally employs a 1-dimensional error diffusion technique. The algorithm calculates the exact difference between a pixel's true average brightness and the limited shade that was actually applied to it. This remaining error is then carried over and added to the next pixel in the row before its new value is calculated, ensuring the overall brightness of the image is preserved. To prevent visual artifacts, this accumulated error is strictly reset to zero at the start of each new row. Dithering strategically mixes the available shades to create microscopic patterns that deceive the human eye into perceiving a smooth, continuous gradient. This makes an image with a low number of gray shades look significantly more detailed, though it introduces a characteristic speckled noise.

2.4.2 Negative

This operation applies a negative filter to the current image. The new color is calculated by subtracting the pixel's original value from the maximum possible channel value (255). Figure 7 shows an example negative image.



Figure 7: Photo turned into a negative.

2.4.3 Brightness

The brightness panel offers two distinct features. The first allows the user to shift the overall brightness by adding a constant offset to every color channel, using a slider that ranges from -255 to 255. Alternatively, the image's brightness can be mapped to a new target range defined by minimum (N1) and maximum (N2) values, both selectable between 0 and 255. This operation effectively stretches or compresses the image's brightness histogram to fit within the newly specified limits. Figure 8 shows photos with brightness changed using the algorithms.



(a) Averaging



(b) Luminance



(c) Desaturation



(d) Maximum decomposition



(e) Minimum decomposition



(f) Only red channel



(g) Only green channel



(h) Only blue channel



(i) Custom using 10 shades



(j) Custom with dithering using 10 shades

Figure 6: Photos grayscaled using the available algorithms.



(a) Brightness with an added offset (b) Brightness with a subtracted offset (c) Brightness range changed to (40, 120)

Figure 8: Photos with changed brightness.

2.4.4 Contrast

The contrast panel offers gamma and log contrast corrections. Gamma correction algorithm applies a non-linear power function to adjust contrast. Each pixel's value is normalized by the image's maximum brightness, raised to the power of the chosen gamma factor (α), and scaled back to the 0-255 range. The gamma factor can be chosen from 0 to 5. The range is capped at 5 because higher values mathematically crush almost all midtones and shadows into pure black, rendering the image unreadable. Logarithmic correction algorithm applies a natural logarithm function to each pixel, which is then normalized against the logarithm of the image's maximum brightness. Because a logarithmic curve rises steeply at lower values, this method effectively expands dark shadows to reveal hidden details without overexposing the already bright areas of the image. Figure 9 shows photos with applied gamma and log corrections.



(a) Gamma correction with $\alpha = 3$ (b) Gamma correction with $\alpha = 0.6$ (c) Log correction

Figure 9: Photos with changed contrast.

2.4.5 Binarization

The binarization panel provides six segmentation algorithms (Otsu, Niblack, Bernsen, Multi-Otsu, Custom multi-threshold, and Custom threshold) along with their respective configuration options. Since this process requires a grayscale image, a dedicated conversion button is included on the panel. These algorithms calculate optimal thresholds to divide the image into distinct brightness classes. In single-threshold methods, pixels exceeding the calculated value become white, while the rest turn black. The chosen operation is executed by clicking the 'Apply' button. Figure 10 shows examples of photos binarized using the algorithms after applying luminance grayscaleing.

Available algorithms:

- **Otsu** - this algorithm finds the optimal threshold value by analyzing the variance in pixel brightness. For every possible threshold value t (from 0 to 255), it divides the image's pixels into two distinct classes: background (pixels below or equal to t) and foreground (pixels above t). For both classes, it calculates their weights (probabilities of a pixel belonging to a class) and their average brightness. Finally, the algorithm computes the between-class variance and selects the threshold t that maximizes this variance, ensuring the best possible separation between dark and light areas.
- **Niblack** - this algorithm is a local adaptive thresholding method, meaning it calculates a unique threshold for every individual pixel rather than applying a single global value. It allows the user to choose the window size and a k parameter that adjusts the threshold's sensitivity. The window slides across the image and the algorithm computes the local mean (μ) and local standard deviation (σ) of the pixels within that specific neighborhood. The threshold (T) for the center pixel is then determined using the formula $T = \mu + k \cdot \sigma$. Because it adapts to its immediate surroundings, this approach is highly effective for segmenting images with uneven illumination, shadows, or varying background contrast. However, its primary drawback is poor performance in flat, uniform regions, where standard deviation is close to 0. This often generates false contours and prominent structural artifacts, as can be seen in Figure 10.

- **Bernsen** - similarly to the Niblack method the thresholds are calculated uniquely for each pixel using a sliding window. The user can select its size and a contrast limit. A higher value keeps uniform regions clean and solid, while a low limit might force the algorithm to unnecessarily binarize tiny noise artifacts. The algorithm finds minimum and maximum values for each window. If the range is greater than selected contrast limit, the pixel is thresholded against the local midpoint. However, if the contrast is lower than the limit - indicating a flat, homogeneous area - the algorithm ignores local variations and uses the absolute gray middle value (128) as the threshold. This algorithm is significantly more sensitive to isolated noise spikes and often results in sharper edges than Niblack's approach.
- **Multi-Otsu** - this algorithm allows the user to specify between 3 and 5 distinct brightness classes, effectively segmenting the image into multiple gray levels ranging from pure black to pure white. It functions as an extension of the basic Otsu method by determining multiple optimal thresholds simultaneously. To find the absolute best separation, the algorithm recursively generates and evaluates every possible combination of thresholds. Because the time complexity grows exponentially with each additional threshold, the maximum number of classes is strictly limited to 5 to ensure the program remains responsive.
- **Custom multi-threshold** - instead of finding the optimal values, two thresholds can be selected by the user, and the image is segmented into black, gray and white.
- **Custom threshold** - instead of finding one optimal threshold, it can be selected by the user, and the image is segmented into black and white.

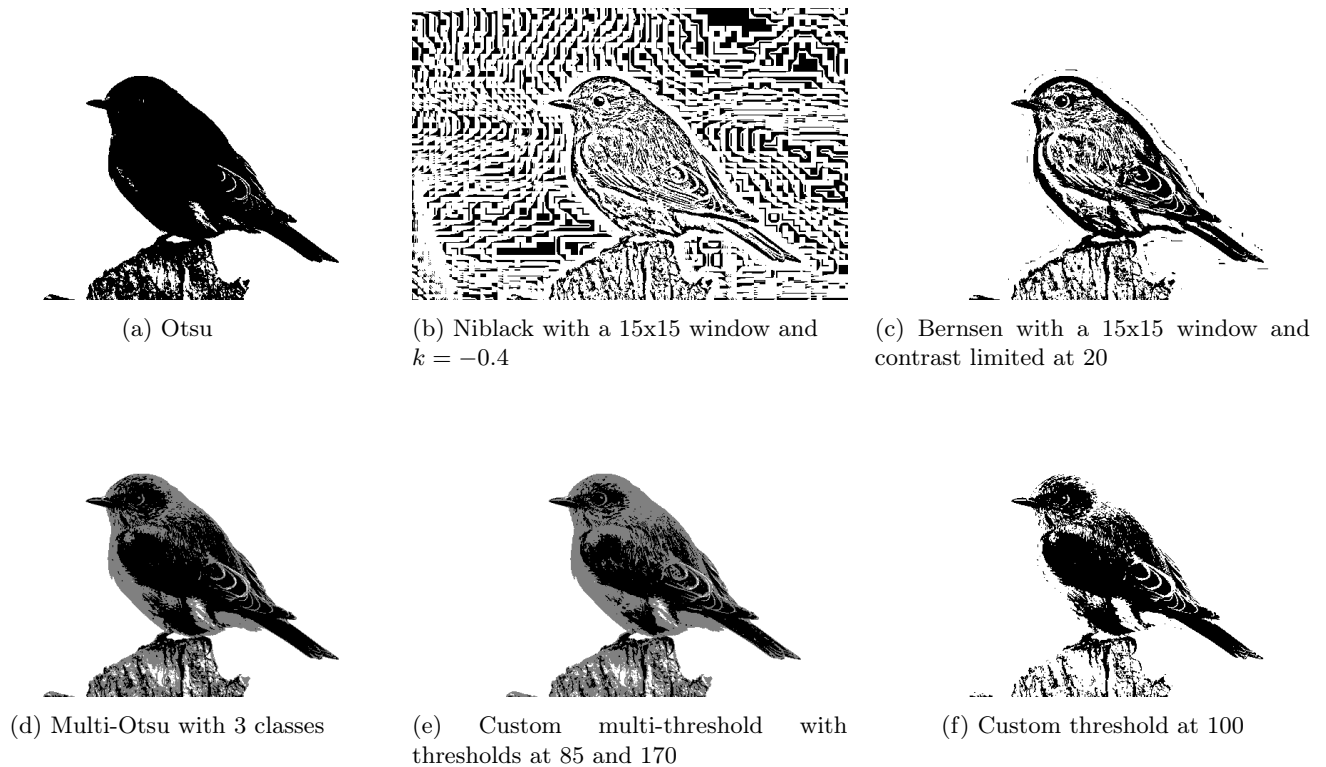


Figure 10: Photos binarized using the available algorithms after luminance grayscaleing.

2.5 Graphic filters

Convolution filters (also known as kernels) modify the image by applying a small matrix to each pixel, in the way that the values of the pixel and its neighbors are multiplied by the corresponding values (weights) in the kernel, the results are summed together, and divided by the sum of the weights; the resulting value then replaces the pixel in the center of the mask, to which the mask was applied. This can be used for any operation which requires context about the pixel's surroundings, instead of just its individual value. The global settings applying to all of those filters (i.e. how they handle situations where in order to apply a mask to some particular pixel near the image border they need to know the values of pixels outside of the image) are further described in section 2.7.

2.5.1 Blurring

A blur effect on an image can be achieved by applying some kind of an averaging mask, i.e. a kernel which applies equal and positive weights to the surrounding pixels. Our application supports two versions of this basic concept.

Box Blur uses a kernel which consists of equal weights; the strength of the blurring effect depends on the size

of the mask used. Additionally, the blurring effect can be weakened by increasing the weight of the center element of the kernel. Figure 11 depicts examples of images modified using different kernels, along with the kernels used.

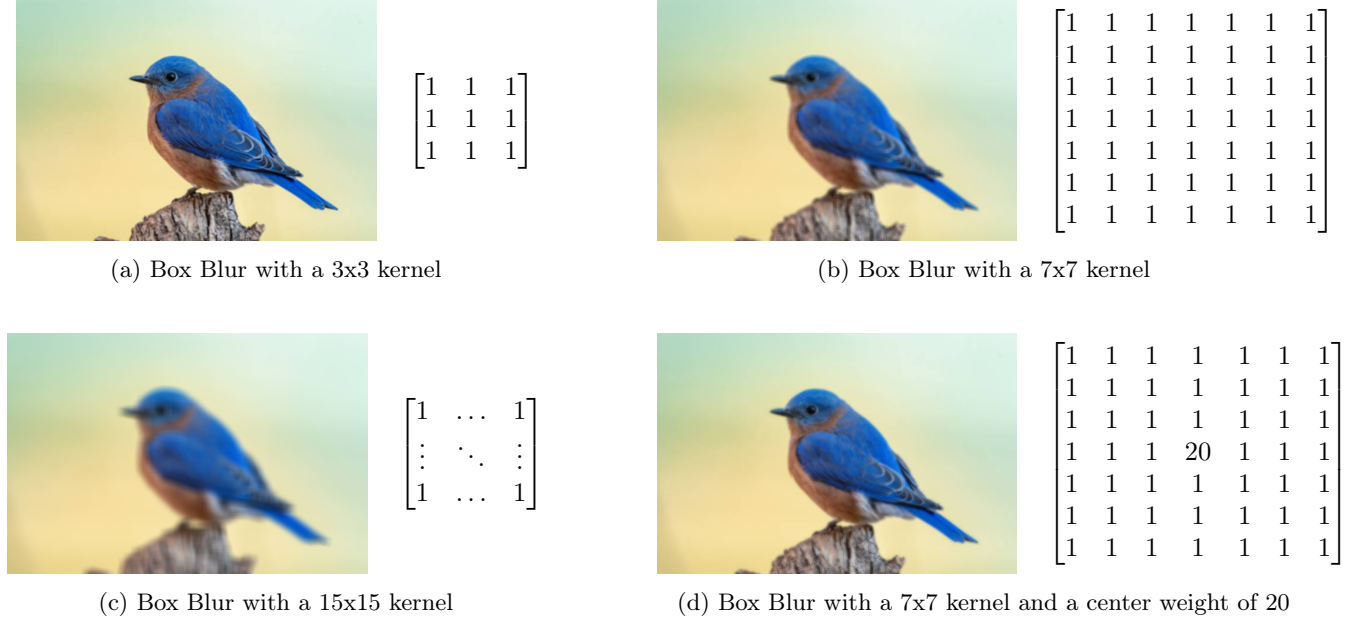


Figure 11: Box Blur performed using various kernels.

Gaussian Blur uses a kernel which approximates a Gaussian distribution of weights around the center pixel. In this case, $G(x, y) = \frac{1}{2\pi\sigma^2} \cdot \exp\left(-\frac{x^2+y^2}{2\sigma^2}\right)$, where G - the weight of a particular pixel, x, y - horizontal and vertical distance from that pixel to the center, σ - the standard deviation of that 2-dimensional normal distribution. The parameter which decides about the strength of the blurring effect is σ , which can be manually entered by the user (and has to be a positive number - in the case of our app, it is constrained to be between 0.1 and 10). In this case, the size of the kernel can't manually be chosen by the user - instead, it is set to be equal to $\lceil 6\sigma \rceil$, since in a 2-dimensional normal distribution almost 99% of the data falls within a circle with a radius of 3σ . Example images modified with Gaussian Blur can be found on Figure 12.



Figure 12: Examples of Gaussian Blur with various standard deviations.

2.5.2 Sharpening

Our app supports two customizable image sharpening algorithms.

Laplacian-based sharpening uses a Laplacian kernel in order to amplify the details in an image. There are two versions available: Standard, which uses the kernel

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix},$$

and Strong, which uses the kernel

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}.$$

By default, the values calculated using the Laplacian kernel are then added back to their original values, resulting in an image whose details are amplified. There are three additional customization options available to the user:

- **Laplacian of Gaussian** allows the user to use Gaussian Blur (with a value of σ also chosen by the user) before performing regular Laplacian-based sharpening, which minimizes the risk of the sharpening mask amplifying random noise rather than genuine details of the picture.
- The user can also choose an **Edge Threshold T** , which is used in order to not amplify random noise in flat areas of the picture. After calculating the Laplacian L , its absolute value on each pixel is then compared against the edge threshold, and values below the threshold are removed before adding the result back to the original picture.
- The **sharpening strength slider** allows the user to choose a value s between 0 and 1, by which the result of the Laplacian kernel is multiplied before adding it back to the original image.

Examples of Laplacian-based sharpening are presented on Figure 13.



(a) Laplacian-based sharpening with the standard kernel and $s = 0.5$



(b) Laplacian-based sharpening with the strong kernel and $s = 0.5$



(c) Laplacian-based sharpening with the standard kernel and $s = 1$



(d) Laplacian-based sharpening with the strong kernel and $s = 1$



(e) Laplacian-based sharpening with the strong kernel, $s = 0.5$, using Laplacian of Gaussian with $\sigma = 1.4$



(f) Laplacian-based sharpening with the strong kernel, $s = 1$, using Laplacian of Gaussian with $\sigma = 1.4$, and an Edge Threshold $T = 20$

Figure 13: Examples of Laplacian-based sharpening with various parameters.

Unsharp masking is an algorithm that's often used in commercial software. In order to sharpen an image, it starts with using Gaussian Blur on the original image. Then, it subtracts the blurred image from the original one - the outcome of this operation is a version of the image that only contains the 'small details' that are normally removed by Gaussian Blur. Finally, the outcome of this whole operation is calculated as

$$Outcome = I + s \cdot (I - B),$$

where I - the original image, s - the sharpening strength (which is a number between 0 and 1 that's chosen by the user), B - the original image blurred using Gaussian Blur (with a standard deviation of σ , which is also chosen by the user). Additionally, the user can also set up an Edge Threshold T , which works in a similar way for both Laplacian-based sharpening and Unsharp Masking. In this case,

$$Mask = \begin{cases} s \cdot (I - B), & |I - B| \geq T \\ 0, & |I - B| < T \end{cases},$$

and the final result is calculated as

$$Outcome = I + Mask.$$

Examples of Unsharp Masking can be found on Figure 14.



(a) Unsharp Masking with $\sigma = 1.4$, $s = 0.5$, $T = 0$



(b) Unsharp Masking with $\sigma = 2$, $s = 1$, $T = 20$

Figure 14: Examples of Unsharp Masking with various parameters.

2.5.3 Edge Detection

Our app implements 5 of the standard, kernel-based edge detection algorithms. The general idea behind most of them is to use different masks for detecting edges in different directions, and then combine their results.

The first relevant group of algorithms are the ones which use 2 kernels, which are responsible for detecting edges in one direction each.

- **Roberts Cross** consists of two small 2x2 kernels, used for detecting diagonal edges in both directions, which are applied to the pixel in their upper-left corner. The kernels are as follows:

$$G_X = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}, \quad G_Y = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}.$$

- The 2-directional **Sobel** operator uses the same idea, but with two 3x3 masks responsible for detecting horizontal and vertical edges. In this case, the kernels are as follows:

$$G_X = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad G_Y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}.$$

For both of those algorithms, the results from both their masks are later combined into one final result, which (for a given pixel a) can be represented as $\sqrt{G_X^2(a) + G_Y^2(a)}$. Examples of those operators can be seen on Figure 15.



(a) Edge detection using Roberts Cross



(b) Edge detection with the Sobel operator

Figure 15: Edge detection with 2-directional algorithms.

Another group of algorithms are the 'compass' ones, which detect edges in 8 directions instead of 2.

- **Prewitt Compass** uses kernels in which the weights 'rotate' along the central pixel in order to detect edges in various directions. The masks for this operator are the following:

$$M_1 = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}, \quad M_2 = \begin{bmatrix} 0 & 1 & 1 \\ -1 & 0 & 1 \\ -1 & -1 & 0 \end{bmatrix}, \quad M_3 = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}, \quad \dots, \quad M_8 = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 0 & -1 \\ 0 & -1 & -1 \end{bmatrix}$$

- **Sobel Compass** is a slightly stronger version of the Prewitt Compass, with kernels such as the following:

$$M_1 = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}, \quad M_2 = \begin{bmatrix} 0 & 1 & 2 \\ -1 & 0 & 1 \\ -2 & -1 & 0 \end{bmatrix}, \quad M_3 = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad \dots, \quad M_8 = \begin{bmatrix} 2 & 1 & 0 \\ 1 & 0 & -1 \\ 0 & -1 & -2 \end{bmatrix}$$

Both of these operators calculate the final result for a given pixel a as $\max(|M_1(a)|, \dots, |M_8(a)|)$. Examples of those operators can be seen on Figure 16.



(a) Edge detection using Prewitt Compass



(b) Edge detection using Sobel Compass

Figure 16: Edge detection with 8-directional algorithms.

Lastly, our app also implements the **Laplace** operator, which is also used for sharpening algorithms. It only uses one kernel:

$$G = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix},$$

which it simply passes through the entire image. An example of this operator can be seen on Figure 17.

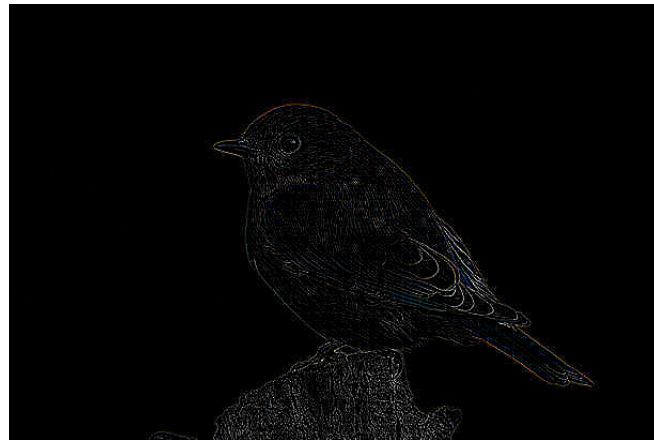


Figure 17: Edge detection with Laplace operator.

Additionally, our app also implements the **Canny Edge Detection Algorithm**, which is a complex 5-step algorithm used in professional image processing contexts. The steps are as follows:

1. **Gaussian Blur**: The image is blurred with a regular Gaussian Blur in order to reduce noise; the σ value can be chosen manually by the user.

2. **Sobel operator:** The 2-directional Sobel operator is used in order to calculate 2 values. The first of them is magnitude, calculated using the $\sqrt{G_X^2 + G_Y^2}$ formula, which tells us how strong of an edge is there on a given pixel. The second of them is the edge's angle, calculated using the $\arctan\left(\frac{G_Y}{G_X}\right)$ formula, which represents the gradient of the edge: it tells us the angle of a line that is perpendicular to the edge in a given point. That value is then rounded to the nearest 45° axis, resulting in one of four values: $0^\circ, 45^\circ, 90^\circ, 135^\circ$.
3. **Non-maximum suppression (edge thinning):** This is a step performed in order to thin each edge to the width of exactly 1 pixel. In order to achieve this, we check the neighbors of each pixel in the direction defined by the edge's angle; if the magnitude calculate in a given pixel is smaller than the magnitude in either of its neighbors, it is suppressed (reduced to zero).
4. **Double thresholding:** Each pixel is then classified into one of three classes depending on its edge magnitude. The user can set 2 thresholds: high and low. If the magnitude value in a given pixel is lower than the low threshold, it is discarded (reduced to zero). If it is higher than the low threshold, but lower than the high threshold, it is classified as a weak edge. If it is higher than the high threshold, it is classified as a strong edge.
5. **Edge tracking by hysteresis:** This is a step performed in order to connect the detected 'strong edges', so that the resulting edges are solid lines that don't randomly break. In a loop, we go through each pixel in the picture. For each 'weak edge', if any of its 8 neighbors is a 'strong edge', the 'weak edge' gets reclassified to a 'strong edge' as well. This process is repeated until no more edges are promoted in a loop iteration; after finishing the loop, are the remaining weak edges are discarded. This results in a final image with clean edge lines, an example of which can be seen on Figure 18.



Figure 18: Edge detection with Canny algorithm.

2.5.4 Custom Mask

This section allows the user to choose their own custom kernel to use as a convolution filter. The size of this kernel can be any odd number of pixels per side that falls between 3x3 and 9x9; the weights for particular pixels are manually entered by the user. Figure 19 shows examples of such custom kernels.



(a) Image transformed with a custom 3x3 kernel

$$\begin{bmatrix} 0 & 0 & 0 \\ 2 & -1 & 1 \\ 0 & 0 & 0 \end{bmatrix}$$



(b) Image transformed with a custom 5x5 kernel

$$\begin{bmatrix} 1 & 0 & 4 & 0 & 1 \\ 0 & 0 & 2 & 0 & 0 \\ -4 & -2 & 1 & 2 & 4 \\ 0 & 0 & -2 & 0 & 0 \\ 1 & 0 & -4 & 0 & 1 \end{bmatrix}$$

Figure 19: Results of using various custom kernels.

2.6 Morphology operations

There are four morphological operations (erosion, dilation, opening, and closing) available under the 'Edit' menu. For each of them, the user can define a custom structuring element by toggling active cells in a grid sized from 3x3 up to 9x9. This element acts as a geometric probe that slides across the image, determining how the underlying pixels are analyzed and transformed. Those operations are primarily used to filter out noise, extract specific image components,

fill microscopic holes, and connect or separate distinct objects. Figure 20 shows examples of photos after morphology operations.

2.6.1 Erosion

Erosion is primarily used to remove pixels from the boundaries of bright objects, effectively shrinking them. This makes the operation useful for eliminating noise, smoothing object contours, and disconnecting weakly joined objects. Operationally, the algorithm slides the user-defined structuring element across the image. For each target pixel, it identifies the darkest pixel within the neighborhood defined by the mask and assigns this minimum brightness value to the target. Although erosion is most commonly used on grayscale images, by converting the RGB values to the HSV color space, the algorithm can also be applied to color images while safely preserving the original hues and saturation.

2.6.2 Dilation

Dilation is primarily used to add pixels to the boundaries of bright objects, effectively expanding them. This makes the operation useful for eliminating dark noise, filling small holes or gaps within objects, and connecting closely spaced or broken components. Operationally, it uses the same algorithm as erosion, with the key difference being that it identifies the brightest pixel instead of the darkest and assigns that brightness value to the target.

2.6.3 Opening

Opening is primarily used to remove small bright artifacts and noise from an image while preserving the overall size and shape of larger objects. It is also highly effective at smoothing object contours and breaking narrow connections between components. The algorithm is simply applying an erosion followed immediately by a dilation using the exact same structuring element.

2.6.4 Closing

Closing is primarily used to fill small holes and gaps within bright objects, and to eliminate small dark artifacts while preserving the overall size and shape of the main components. It is also highly effective at smoothing object contours by filling in narrow gaps between closely spaced objects. The algorithm is simply applying a dilation followed immediately by an erosion using the exact same structuring element.



(a) Erosion using a 3x3 square element



(b) Dilation using a 3x3 square element



(c) Opening using a 3x3 square element



(d) Closing using a 3x3 square element



(e) Erosion using a 3x3 cross element after luminance grayscaling



(f) Dilation using a 3x3 cross element after luminance grayscaling

Figure 20: Photos after using available morphology operations.

2.7 Settings

The settings menu allows the user to change the default way in which convolution filters deal with pixels near image boundaries, where in order to apply the mask to some particular pixel, it is necessary to know the values of pixels that

are outside of the image. Upon selection, the chosen option applies to all filters. The user can choose one of seven options:

1. **Crop the image:** When this option is selected, the app never applies the mask to pixels for which it would span outside of the picture; instead, it removes all such pixels, resulting in a final image that is smaller than the original - the larger the mask used, the smaller the final image. This is the only option that is entirely mathematically precise without requiring any additional assumptions. However, it is also quite impractical. Its main drawback is that it removes information about the original image, which most users don't want. Additionally, the original image deteriorates further with all subsequent operations, rendering multi-step image processing pipelines entirely impossible without significant data loss.
2. **Keep original pixels:** When this option is selected, the pixels to which the mask can't be applied due to lack of information about pixels outside of the image boundary are kept intact. This results in the image consisting of two very different parts: the processed center and the unprocessed boundary pixels, which is also quite impractical. One obvious benefit of this approach is the added opportunity to perform long chains of operations without losing data at each step - however, while the pixels near image boundary aren't lost, they're still negatively affecting the result of subsequent operations on the pixels bordering them, thus still letting some negative effects spread further away from the image boundary with every subsequent operation.
3. **Assume outside is black:** When this option is selected, all pixels outside of the image are assumed to be black - i.e. have a value of (0, 0, 0). This results in the formation of a dark 'halo' around the image. An obvious benefit of this approach is the clear visual implication that the image boundary shouldn't be considered precise and/or important, which treats the user with a relatively high level of interpretative honesty. The main drawbacks include the unnatural look of the processed image, as well as the dark 'halo' spreading away from the image boundary with every subsequent operation, thus deteriorating a larger part of the image.
4. **Assume outside is white:** Works analogously to the 'assume outside is black' option, with the exact same benefits and drawbacks. In this option, the described 'halo' around the image is white (instead of black). The choice between these options is just a matter of preference.
5. **Assume outside is gray:** Works analogously to the 'assume outside is black' option, with the exact same benefits and drawbacks. In this option, the described 'halo' around the image is gray (instead of black). The choice between these options is just a matter of preference.
6. **Copy outer-most pixel:** This is the default option. Whenever it needs to compute a value of a pixel that is outside of the image boundary, it assumes it to be the same as the value of its closest pixel within the image. This results in mostly natural-looking pictures; however, near the edge of the picture some features displayed in the picture might appear artificially elongated, especially when using large masks.
7. **Mirror outer-most pixels:** This variant attempts to deal with the problem of artificially stretched out artifacts on the image border by mirroring the outermost pixels instead of just copying the border pixel; for each pixel outside of the picture, this version of the algorithm assumes that it is of the same value as a pixel that is symmetric to it (with the border of the picture acting as the axis of symmetry). This can result in unnatural results as well (in cases where the supposed extension of the picture can't intuitively be symmetric to the objects within the picture), but it is quite useful for working with pictures containing a pattern rather than some structure with a strictly asymmetric shape.

All of the above options are illustrated on figure 21, with a 21x21 pixel Box Blur mask.



(a) Crop the image



(b) Keep original pixels



(c) Assume outside is black



(d) Assume outside is white



(e) Assume outside is gray



(f) Copy outer-most pixel



(g) Mirror outer-most pixels

Figure 21: Regular Box Blur used with all of the different border handling settings.